

**Figure 4 Output vs. input of two LN layers, with tensor elements colored to indicate different channel and token dimensions.** The input tensor has a shape of (samples, tokens, and channels), with elements visualized by assigning consistent colors to the same tokens (left two panels) and channels (right two panels). *Left two panels:* points representing the same token (same color) form straight lines across different channels, as LN operates linearly across channels for each token. Interestingly, when plotted collectively, these lines form a non-linear tanh-shaped curve. *Right two panels:* each channel’s input spans different ranges on the  $x$ -axis, contributing distinct segments to the overall tanh-shaped curve. Certain channels (e.g., red, green, and pink) exhibit more extreme  $x$  values, which are squashed by LN.

linear operations. LN normalizes in a per-token manner, only linearly transforming each token’s activations. As tokens have different mean and standard deviation values, the linearity does not hold collectively on all activations of the input tensor. Nonetheless, it is still surprising to us that the actual non-linear transformation is highly similar to a scaled tanh function.

For such an  $S$ -shaped curve, we note that the central part, represented by points with  $x$  values close to zero, is still mainly in a linear shape. Most points ( $\sim 99\%$ ) fall in this linear range. However, there are still many points that clearly fall out of this range, which are considered to have “extreme” values, e.g., those with  $x$  larger than 50 or smaller than -50 in the ViT model. Normalization layers’ main effect for these values is to *squash* them into less extreme values, more in line with the majority of points. This is where normalization layers could not be approximated by a simple affine transformation layer. We hypothesize this non-linear and disproportional squashing effect on extreme values is what makes normalization layers important and indispensable.

Recent findings by [Ni et al. \(2024\)](#) similarly highlight the strong non-linearities introduced by LN layers, demonstrating how the non-linearity enhances a model’s representational capacity. Moreover, this squashing behavior mirrors the saturation properties of biological neurons for large inputs, a phenomenon first observed about a century ago ([Adrian, 1926](#); [Adrian and Zotterman, 1926a,b](#)).

**Normalization by tokens and channels.** How does an LN layer perform a linear transformation for each token but also squash the extreme values in such a non-linear fashion? To understand this, we visualize the points grouped by tokens and channels, respectively. This is plotted in Figure 4 by taking the second and third subplots for ViT from Figure 2, but with a sampled subset of points for more clarity. When we select the channels to plot, we make sure to include the channels with extreme values.

On the left two panels of Figure 4, we visualize each token’s activations using the same color. We observe that all points from any single token do form a straight line. However, since each token has a different variance, the slopes are different. Tokens with smaller input  $x$  ranges tend to have smaller variance, and the normalization layer will divide their activations using a smaller standard deviation, hence producing a larger slope in the straight line. Collectively, they form an  $S$ -shaped curve that resembles a tanh function. In the two panels on the right, we color each channel’s activations using the same color. We find that different channels tend to have drastically different input ranges, with only a few channels (e.g., red, green, and pink) exhibiting large extreme values. These are the channels that get squashed the most by the normalization layer.

## 4 Dynamic Tanh (DyT)

Inspired by the similarity between the shapes of normalization layers and a scaled tanh function, we propose Dynamic Tanh (DyT) as a drop-in replacement for normalization layers. Given an input tensor  $\mathbf{x}$ , a DyT layer is defined as follows:

$$\text{DyT}(\mathbf{x}) = \gamma * \tanh(\alpha \mathbf{x}) + \beta \quad (2)$$

where  $\alpha$  is a learnable scalar parameter that allows scaling the input differently based on its range, accounting for varying  $x$  scales (Figure 2). This is also why we name the whole operation “Dynamic” Tanh.  $\gamma$  and  $\beta$  are learnable, per-channel vector parameters, the same as those used in all normalization layers—they allow the output to scale back to any scales. This is sometimes considered a separate affine layer; for our purposes, we consider them to be part of the DyT layer, just like how normalization layers also include them. See Algorithm 1 for implementation of DyT in Pytorch-like pseudocode.

Integrating DyT layers into an existing architecture is straightforward: one DyT layer replaces one normalization layer (see Figure 1). This applies to normalization layers within attention blocks, FFN blocks, and the final normalization layer. Although DyT may look like or be considered an activation function, this study only uses it to replace normalization layers without altering any parts of the activation functions in the original architectures, such as GELU or ReLU. Readers interested in the use of harmonic or hyperbolic functions as activation functions can refer to Hashemi et al. (2024). We also observe that there is little need to tune the hyperparameters used by the original architectures for DyT to perform well.

---

### Algorithm 1 Pseudocode of DyT layer.

---

```
# input x has the shape of [B, T, C]
# B: batch size, T: tokens, C: dimension

class DyT(Module):
    def __init__(self, C, init_alpha):
        super().__init__()
        self.alpha = Parameter(ones(1) * init_alpha)
        self.gamma = Parameter(ones(C))
        self.beta = Parameter(zeros(C))

    def forward(self, x):
        x = tanh(self.alpha * x)
        return self.gamma * x + self.beta
```

---

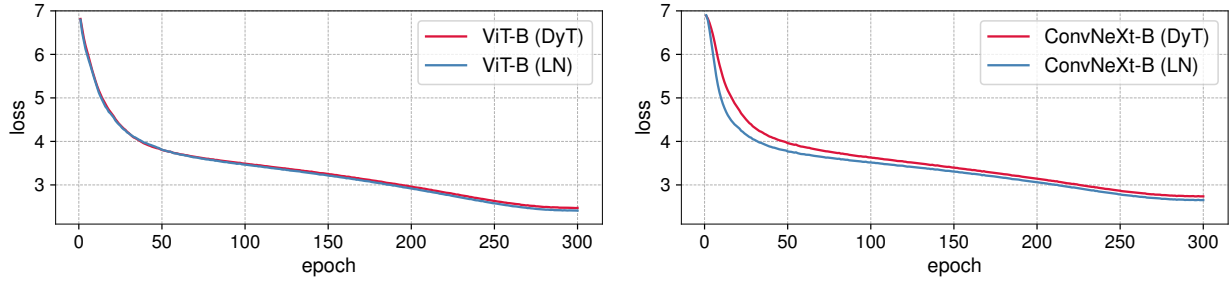
**On scaling parameters.** We always simply initialize  $\gamma$  to an all-one vector and  $\beta$  to an all-zero vector following normalization layers. For the scalar parameter  $\alpha$ , a default initialization of 0.5 is generally sufficient, except for LLM training. A detailed analysis of  $\alpha$  initialization is provided in Section 7. Unless explicitly stated otherwise,  $\alpha$  is initialized to 0.5 in our subsequent experiments.

**Remarks.** DyT is *not* a new type of normalization layer, as it operates on each input element from a tensor independently during a forward pass without computing statistics or other types of aggregation. It does, however, preserve the effect of normalization layers in squashing the extreme values in a non-linear fashion while almost linearly transforming the very central parts of the input.

## 5 Experiments

To demonstrate the effectiveness of DyT, we experiment with Transformers and a few other modern architectures across a diverse range of tasks and domains. In each experiment, we replace the LN or RMSNorm in the original architectures with DyT layers and follow the official open-source protocols to train and test both versions of the models. Detailed instructions for reproducing our results are provided in Appendix A. Notably, to highlight the simplicity of adapting DyT, we use hyperparameters identical to those utilized by the normalized counterparts. For completeness, additional experimental results regarding tuning of learning rates and initial values of  $\alpha$  are provided in Appendix B.

**Supervised learning in vision.** We train Vision Transformer (ViT) (Dosovitskiy et al., 2020) and ConvNeXt (Liu et al., 2022) of “Base” and “Large” sizes on the ImageNet-1K classification task (Deng et al., 2009). These models are selected due to their popularity and distinct operations: attention in ViT and convolution in ConvNeXt. Table 1 reports the top-1 classification accuracies. DyT performs slightly better than LN across both architectures and model sizes. We further plot the training loss for ViT-B and ConvNeXt-B in Figure 5. The curves show that the convergence behaviors of DyT and LN-based models are highly aligned.



**Figure 5 Training loss curves for ViT-B and ConvNeXt-B models.** The loss curves for both model types exhibit similar patterns between LN and DyT, suggesting that LN and DyT may share similar learning dynamics.

| model      | LN    | DyT   | change           |
|------------|-------|-------|------------------|
| ViT-B      | 82.3% | 82.5% | $\uparrow 0.2\%$ |
| ViT-L      | 83.1% | 83.6% | $\uparrow 0.5\%$ |
| ConvNeXt-B | 83.7% | 83.7% | -                |
| ConvNeXt-L | 84.3% | 84.4% | $\uparrow 0.1\%$ |

**Table 1 Supervised classification accuracy on ImageNet-1K.** DyT achieves better or similar performance than LN across both architectures and model sizes.

**Self-supervised learning in vision.** We benchmark with two popular visual self-supervised learning methods: masked autoencoders (MAE) (He et al., 2022) and DINO (Caron et al., 2021). Both by default use Vision Transformers as the backbones, but have different training objectives: MAE is trained with a reconstruction loss, and DINO uses a joint-embedding loss (LeCun, 2022). Following the standard self-supervised learning protocol, we first pretrain models on ImageNet-1K without using any labels and then test the pretrained models by attaching a classification layer and fine-tuning them with labels. The fine-tuning results are presented in Table 2. DyT consistently performs on par with LN in self-supervised learning tasks.

| model                      | LN    | DyT   | change             |
|----------------------------|-------|-------|--------------------|
| MAE ViT-B                  | 83.2% | 83.2% | -                  |
| MAE ViT-L                  | 85.5% | 85.4% | $\downarrow 0.1\%$ |
| DINO ViT-B (patch size 16) | 83.2% | 83.4% | $\uparrow 0.2\%$   |
| DINO ViT-B (patch size 8)  | 84.1% | 84.5% | $\uparrow 0.4\%$   |

**Table 2 Self-supervised learning accuracy on ImageNet-1K.** DyT performs on par with LN across different pretraining methods and model sizes in self-supervised learning tasks.

**Diffusion models.** We train three Diffusion Transformer (DiT) models (Peebles and Xie, 2023) of sizes B, L and XL on ImageNet-1K (Deng et al., 2009). The patch size is 4, 4, and 2, respectively. Note that in DiT, the LN layers’ affine parameters are used for class conditioning in DiT, and we keep them that way in our DyT experiments, only replacing the normalizing transformation with the  $\tanh(\alpha x)$  function. After training, we evaluate the Fréchet Inception Distance (FID) scores using the standard ImageNet “reference batch”, as presented in Table 3. DyT achieves comparable or improved FID over LN.

| model  | LN   | DyT  | change           |
|--------|------|------|------------------|
| DiT-B  | 64.9 | 63.9 | $\downarrow 1.0$ |
| DiT-L  | 45.9 | 45.7 | $\downarrow 0.2$ |
| DiT-XL | 19.9 | 20.8 | $\uparrow 0.9$   |

**Table 3 Image generation quality (FID, lower is better) on ImageNet.** DyT achieves comparable or superior FID scores to LN across various DiT model sizes.